



Chinese Society of Aeronautics and Astronautics
& Beihang University

Chinese Journal of Aeronautics

cja@buaa.edu.cn
www.sciencedirect.com



FULL LENGTH ARTICLE

Implicit geometry neural network for mesh generation



Ran XU^{a,b}, Hongqiang LYU^{c,*}, Jian YU^{a,b}, Chenyu BAO^c, Hongfei WANG^{a,b},
Yufei LIU^c, Xuejun LIU^{a,b,*}

^a College of Artificial Intelligence, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China

^b The Key Laboratory of Brain-Machine Intelligence Technology, Ministry of Education, Nanjing 211106, China

^c College of Aerospace Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China

Received 13 March 2024; revised 12 August 2024; accepted 9 October 2024

Available online 24 November 2024

KEYWORDS

Mesh generation;
Implicit geometry;
Mesh size function;
Geometric features;
Generative adversarial
learning

Abstract The accuracy of numerical computation heavily relies on appropriate meshing, which serves as the foundation for numerical computation. Although adaptive refinement methods are available, an adaptive numerical solution is likely to be ineffective if it originates from a poorly initial mesh. Therefore, it is crucial to generate meshes that accurately capture the geometric features. As an indispensable input in meshing methods, the Mesh Size Function (MSF) determines the quality of the generated mesh. However, the current generation of MSF involves human participation to specify numerous parameters, leading to difficulties in practical usage. Considering the capacity of machine learning to reveal the latent relationships within data, this paper proposes a novel machine learning method, Implicit Geometry Neural Network (IGNN), for automatic prediction of appropriate MSFs based on the existing mesh data, enabling the generation of unstructured meshes that align precisely with geometric features. IGNN employs the generative adversarial theory to learn the mapping between the implicit representation of the geometry (Signed Distance Function, SDF) and the corresponding MSF. Experimental results show that the proposed method is capable of automatically generating appropriate meshes and achieving comparable meshing results compared to traditional methods. This paper demonstrates the possibility of significantly decreasing the workload of mesh generation using machine learning techniques, and it is expected to increase the automation level of mesh generation.

© 2024 The Authors. Published by Elsevier Ltd on behalf of Chinese Society of Aeronautics and Astronautics. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

* Corresponding authors.

E-mail addresses: hongqiang.lu@nuaa.edu.cn (H. LYU), xuejun.liu@nuaa.edu.cn (X. LIU).

Peer review under responsibility of Editorial Committee of CJA



Production and hosting by Elsevier

1. Introduction

Many computational techniques, such as computational modeling and numerical simulation, aim at solving Partial Differential Equations (PDE). Prior to solving the PDE, it is necessary to discretely represent the geometric domain by

<https://doi.org/10.1016/j.cja.2024.11.025>

1000-9361 © 2024 The Authors. Published by Elsevier Ltd on behalf of Chinese Society of Aeronautics and Astronautics. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

dividing it into a mesh composed of simple elements. Unstructured meshes play a crucial role in numerical computation, particularly due to the superior flexibility in varying spatial resolution.¹ Therefore, automatically generating high-quality unstructured meshes is crucial for successfully solving PDEs. A PDE solution is likely to be deemed invalid if it originates from a poor-quality initial mesh.² Moreover, the initial mesh has to be generated solely based on the geometric features of the geometry, without any numerical solution information, prior to solving the problem for the first time. Indeed, a well-designed initial mesh can obviate the necessity for adaptive mesh refinement and enhance the efficacy of numerous adaptive methods. The main objective of this study is to develop a model capable of automatically generating mesh that aligns with the geometric features, i.e. geometrically adaptive meshes.

The main methods for generating unstructured meshes include delaunay,^{3,4} advance-front,^{5,6} and quadtree (octree in 3D).⁷ All these strategies need to specify a Mesh Size Function (MSF), where the value assigned to each point in the MSF indicates the desired element size in its vicinity.^{8,9} By incorporating MSF as input for mesh generation, the meshing process can be divided into two consecutive tasks: (A) establishing a MSF that defines the element sizes, and (B) generating a mesh that is compatible with the given MSF. The meshing process in the second task may require only a short amount of time compared to the time spent on defining the MSF in the first task for complex geometries.¹⁰ The MSF, being an indispensable input for meshing methods, significantly influences the quality of generated meshes. Consequently, the bottleneck in automatic mesh generation lies not in the meshing process itself, but rather in the generation of efficient and rational MSFs. Various approaches can be considered to define geometrically adaptive MSFs, such as utilizing background meshes, specifying spacing at geometric entities, or employing point and linear sources. The source term is generally preferred for complex geometries due to its greater flexibility. Furthermore, the incorporation of source terms can complement the utilization of background meshes, as well as refinement based on geometric entities. However, the process of selecting an appropriate set of sources for a complex geometry is time-consuming and necessitates the specification of numerous parameters which mainly depend on human experience.¹¹

In the field of geometrically adaptive MSF generation, a great amount of work has been conducted. The background mesh-based method has become the predominant research methodology due to its advantages of reduced user interaction and increased flexibility in representing local element resolution.¹² The early representative work¹³ employed a background mesh to specify the MSF, wherein the spacing parameters of the mesh points are distributed across the nodes of a uniform Cartesian grid by solving the Poisson equation. To address memory consumption issues associated with uniform Cartesian grids, some researchers have turned to constructing MSFs using non-uniform Cartesian grids such as octrees. A framework¹⁴ incorporates a background octree grid for storing MSFs, a set of source entities for providing geometry-based size information, and an interpolation module to calculate the size of the background octree grid using the source entities. Similarly, a method⁹ for initializing and smoothing the size function is presented. The gradient limiting equations are solved using the Hamilton-Jacobi solver to

obtain a graded size function that satisfies the necessary gradient constraints. However, these methods have consistently required manual specification of numerous parameters and the incorporation of different priori information for distinct meshing scenarios. Therefore, it is worthwhile to investigate the possibility of automatically generating geometrically adaptive MSF with as little human participation as possible.

The application of deep learning, particularly neural networks, has demonstrated remarkable success in capturing the underlying regulations hidden among data. The utilization of neural networks has garnered significant attention in scientific computing and engineering fields, particularly in the areas of flow field prediction,^{15–17} PDE solving^{18,19}, shape design, and parameter optimization.^{20,21} Limited research^{22–27} has been conducted on the application of neural networks for predicting MSF in mesh generation. Early research proposed mesh generators guided by neural networks, in which the size and location of the elements are determined using MSF provided by neural networks. These models are limited to low-dimensional modeling and do not perform well in dealing with highly nonlinear problems. In addition to the early research in mesh generation, recent noteworthy studies in mesh refinement are MeshingNet,²⁸ GMRNet.²⁹ They employ a fully connected network and a graph convolutional network³⁰ to learn mappings from geometry parameters to MSFs. The methods presented in^{31,32} both utilize the U-Net architecture to generate refined meshes by predicting MSFs in linear elasticity problems and computational fluid dynamics, respectively. These methods based on neural networks attempt to replace the traditional process of obtaining optimal MSFs for mesh refinement. However, they still need a coarse mesh to calculate the initial solution for constructing the MSF training dataset.

The traditional generation of the initial meshes aligned to geometric features is multi-step and complicated. It includes calculating geometric properties, manually specifying key parameters, and constructing and solving equations on a background mesh. Traditional mesh size function methods (e.g., curvature-based, proximity-based, source-based) for processing new geometry profiles not only are time-consuming but also require separately processing of local and global geometric features. We assume that such a multi-step procedure constructs a hidden sophisticated mapping from geometry to corresponding MSF since it is determined by geometric features in principle. As deep learning neural networks are capable of learning highly nonlinear relationships between high dimensional data, this mapping is learnable by deep learning models, with the availability of high-quality MSF data associated with the corresponding geometries. The trained neural network processes new geometry by simply feeding geometry into the neural network and captures both local and global geometry features to generate more accurate meshes. Therefore, we aim to replace this complex mesh generation process with a deep learning model which efficiently maps a given geometry to its corresponding MSF.

In this paper, we propose Implicit Geometry Neural Network (IGNN) that utilizes a Generative Adversarial Network (GAN)³³ architecture to generate geometrically adaptive MSF for unstructured mesh generation. The model consists of the generator and the discriminator, which adversarially learn the hidden mapping between a geometry and its corresponding MSF. The Signed Distance Function (SDF), an implicit geometry representation, is used as input for the

generator to predict the corresponding MSF. The discriminator is employed to assess the authenticity of both the generated MSF and the real MSF. The initially generated MSFs may be of low quality, but as the generator learns from the discriminator, the generated MSFs gradually approach closer to the real ones. Trained on data comprised of randomly generated geometries and their corresponding MSFs, the IGNN is able to capture the mapping between geometries and related MSFs after generative adversarial learning between the generator and the discriminator. The trained IGNN takes the SDF of the new geometry as input to produce an appropriate MSF for generating a mesh aligned to its geometric features. We conduct experiments to demonstrate the efficacy and accuracy of IGNN in capturing the geometric features of the target domain to be meshed.

The remainder of this paper is organized as follows: [Section 2](#) summarizes the fundamental principles underlying the SDF, MSF, and GAN. [Section 3](#) presents the model proposed in this paper and provides a detailed description of the neural network architecture. [Section 4](#) presents and analyzes the experimental results obtained through the proposed method. [Section 5](#) summarizes the conclusions of the presented work and discusses potential future research directions.

2. Background

Given a domain geometry, the objective of this paper is to obtain a geometrically adaptive MSF for unstructured mesh generation. To this end, we propose IGNN, a neural network employing a generative adversarial mechanism that utilizes SDFs as inputs and outputs MSFs. This section presents the fundamental concepts and definitions related to SDF, MSF, and GAN that are utilized in the method proposed in this paper.

2.1. Signed distance function

The signed distance function $\phi(x)$, referred to as SDF, is utilized in this paper as an implicit representation of the geometry. It provides the shortest distance from a point x within the domain to the geometric boundary. In the level set method,³⁴ the geometry boundary is defined as the zero level set $\phi(x) = 0$, where the boundary can be propagated in time by solving the level set eikonal equations. The SDF is defined as follows

$$\phi(x) = \begin{cases} -d(x, \partial\Omega) & \text{if } x \in \Omega \\ 0 & \text{if } x \in \partial\Omega \\ d(x, \partial\Omega) & \text{if } x \in \bar{\Omega}^c \end{cases} \quad (1)$$

where Ω , $\bar{\Omega}^c$, $\partial\Omega$ respectively denote the interior, exterior, and boundary of the geometric domain. Let x be an arbitrary point within the defined domain, where $d(x, \partial\Omega)$ represents the minimum distance from point x to the boundary. The absolute value of the SDF $|\phi(x)|$ represents the minimum distance from a point within the defined domain to its boundary. The sign of $\phi(x)$ is negative inside the geometric domain, zero on the geometry boundary, and positive outside of it. As a unified implicit geometry, SDF can represent not only local features (e.g., curvature, sharp corners), but also global features (e.g.,

geometric contours, shapes) because they are defined globally throughout space. This property helps IGNN to capture local and global features of the geometry.³⁵

In practical application, SDF is discretely represented and in this paper, we utilize a uniform Cartesian grid for its representation. This choice is motivated by the fact that representing SDFs as uniform Cartesian grids in matrix form allows for leveraging the powerful feature learning capabilities of the Convolutional Neural Network (CNN). The SDF on the Cartesian grid implicitly represents a geometric domain, as illustrated in [Fig. 1](#). There exist numerous well-established methods for solving eikonal equations on Cartesian grids to acquire the SDF, and we employ the Fast Marching Method (FMM)³⁶ to obtain a discrete representation of the SDF on a uniform Cartesian grid. Additionally, the SDF possesses the property of $|\nabla\phi| = 1$, ensuring that its gradient is always equal to 1 and thus satisfying Lipschitz continuity. Lipschitz continuity guarantees that even small perturbations in the input data result in only minor changes to the output predictions, thereby ensuring greater stability and consistency in the CNN's predictions. Simultaneously, the incorporation of Lipschitz continuity mitigates the issues of gradient explosion or vanishing and enhances CNN's generalization capacity.³⁷

2.2. Mesh size function

As an essential input for the unstructured mesh generation process, the MSF is defined as a function $h(x)$, a scalar function that represents the expected element size at any given point x within the domain to be meshed. Furthermore, it is necessary to constrain the gradient of the element size in order to ensure a smooth transition between elements throughout the domain. In practical applications, a commonly employed approach involves discretizing the size function on a background mesh and subsequently estimating the actual value of $h(x)$ through interpolation. This is often preferred due to its ability to effectively adjust local element sizes by modifying values defined on a few grid nodes. Similar to the SDF, we discretize the MSF representation into a matrix format on a uniform Cartesian grid for convenience. The MSF on a discrete Cartesian grid and its corresponding unstructured meshes are illustrated in [Fig. 2](#).

Numerous methods are available for generating MSF on uniform Cartesian grids. In this study, we adopt the Poisson equation presented in Refs. [2](#), [13](#) to construct training data for mesh size distributions. This approach is analogous to solving a steady-state heat transfer problem with a discrete heat source, whereby the mesh sizes are distributed across the nodes of the Cartesian background grid. The method defines the Poisson equation as follows, and characterizes the distribution of mesh size through a set of boundary conditions specified by the following equation,

$$\begin{cases} \nabla^2 S = \sum_{n=1}^N \psi_n (SJ_n - I_n) \\ S = S_b \quad \text{on boundary} \end{cases} \quad (2)$$

where S denotes the MSF, S_b is the boundary condition, and $\sum_{n=1}^N \psi_n (SJ_n - I_n)$ is the source term that maintains the mesh sizes. In the source term, N denotes the total number of sources, ψ_n represents the weighting factor. For nodal and linear sources, the functions I_n and J_n are defined as

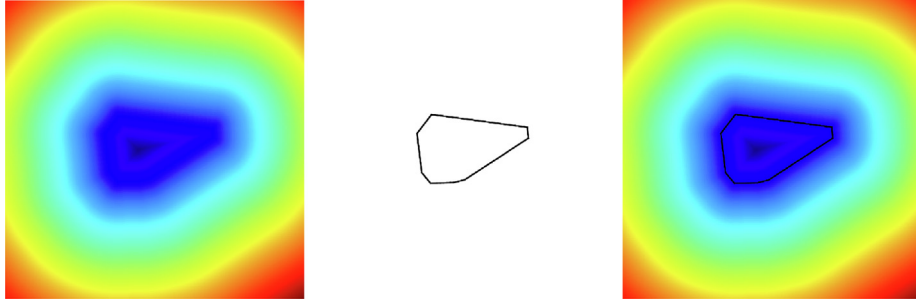


Fig. 1 SDF of discretized representation on a uniform Cartesian grid (left), geometry itself (middle), and zero level set coincide with geometric boundary (right).

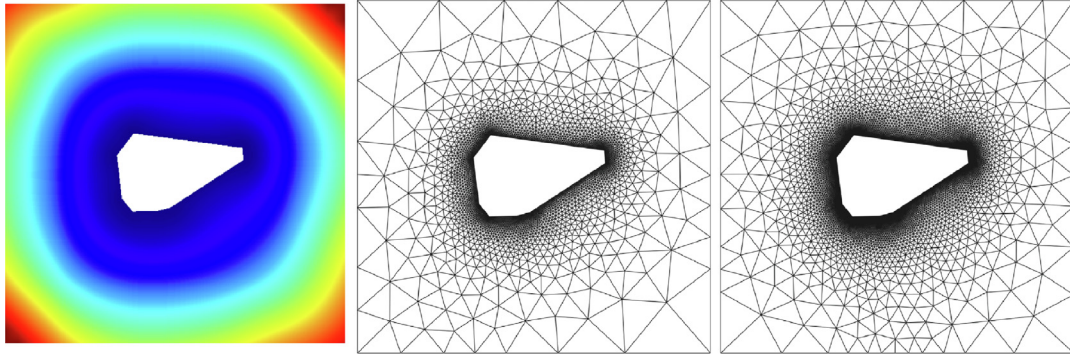


Fig. 2 MSF on a discrete Cartesian grid and its corresponding unstructured meshes (The discretized uniform Cartesian grid representation of the MSF is shown on the left, while the coarse/fine meshes generated at different scales by specifying different maximum/minimum element sizes are depicted in the middle and on the right).

$$I_n = \begin{cases} S_n/r_n^2 & \text{Nodal source} \\ \frac{1}{|\ell_n|} \int_{\ell_n} \frac{f(\ell)}{r(\ell)^2} d\ell & \text{Linear source} \end{cases} \quad (3)$$

$$J_n = \begin{cases} 1/r_n^2 & \text{Nodal source} \\ \frac{1}{|\ell_n|} \int_{\ell_n} \frac{1}{r(\ell)^2} d\ell & \text{Linear source} \end{cases} \quad (4)$$

where r_n represents the Euclidean distance between the background grid point and the n th nodal source, S_n denotes the mesh element size at the n th nodal source, $r(\ell)$ indicates the Euclidean distance from the background grid point to the linear source, and $f(\ell)$ signifies the linear variation of mesh element size along a linear source with length ℓ_n . Nodal and linear sources can be positioned anywhere within the domain, preferably near the boundaries that represent geometric features. These sources are the simplest geometric entities, where size values are defined and stored at the nodal sources or at the endpoints of the linear sources.

2.3. Generative adversarial network

GANs, as generative models in the field of deep learning, have gained significant traction in image synthesis and processing, emerging as a hot research topic in recent years. GANs often comprise two modules: (A) a generator, G , which tries to capture the latent distribution of real samples and generate new data samples; and (B) a discriminator, D , which is commonly a binary classifier that distinguishes between generated and real samples. In order to learn the G 's distribution p_g over data

x , the input noise variable is a priori defined as $p_z(z)$, where z represents random noise. Here, G represents a mapping function $G(z; \theta_g)$ that transforms noise space into data space by taking z as input and utilizing the parameters θ_g . Similarly, D represents a mapping function $D(x; \theta_d)$ that maps data space to scalar space by taking x as input and θ_d as parameters. The output of D , $D(x)$, is a scalar that represents the probability of x being real data rather than generated by G . During simultaneous training, the objective of D is to maximize the log-probability of correctly classifying real data samples $\ln(D(x))$, while the objective of G is to minimize the log-probability of discriminator misclassifying generated data samples as fake $\ln(1 - D(G(z)))$. The training objective function of GANs is as follows,

$$\min_G \max_D V(D, G) = E_{x \sim p_{\text{data}}(x)} [\ln D(x)] + E_{z \sim p_z(z)} [\ln(1 - D(G(z)))] \quad (5)$$

where $p_{\text{data}}(x)$ is the distribution of the real data x and $p_z(z)$ is the prior distribution of the random noise z .

In addition to the aforementioned original GAN, there exist numerous representative variants of GANs.^{38–42} Unlike the original GAN composed of fully connected neural networks, Deep Convolution GAN (DCGAN)³⁸ employs convolution and transposed convolution (DeConvolution) to enable spatial downsampling and upsampling operators to be learned during training. The conditional GAN (cGAN)³⁹ framework extends GAN to conditional settings by introducing additional information y as a condition for both the generator and dis-

criminator. The training objective function of cGAN is as follows,

$$\min_G \max_D V(D, G) = E_{x \sim p_{\text{data}}(x)} [\ln D(x|y)] + E_{z \sim p_z(z)} [\ln(1 - D(G(z|y)))] \quad (6)$$

where y represents additional conditional information, encompassing class labels, text, images, and other relevant features. Pix2Pix⁴¹ aims to offer a general-purpose solution for various image-to-image translation tasks through the use of cGAN. In cGAN, the generator takes random noise z as input along with condition y , whereas in Pix2Pix, the generator solely relies on the condition y . The objective of Pix2Pix is to establish a mapping between an input image y and an output image $G(y)$, such as colorization of black and white images. The objective function of Pix2Pix is as follows,

$$L_{\text{cGAN}}(D, G) + \lambda L_{\text{L1}}(G) \quad (7)$$

which comprises two parts: the cGAN objective function $L_{\text{cGAN}}(D, G)$ and the regression loss $L_{\text{L1}}(G)$ for the generated image, where λ is the hyperparameter. The two parts of the objective function are defined as follows

$$L_{\text{cGAN}}(D, G) = E_{x,y} [\ln D(x, y)] + E_y [\ln(1 - D(G(y), y))] \quad (8)$$

$$L_{\text{L1}} = E_{x,y} [\|x - G(y)\|_1] \quad (9)$$

3. Proposed method

Our purpose is to use neural networks instead of the traditional multi-step approach that relies on human participation to obtain MSFs for mesh generation, as depicted in Fig. 3. Traditional methods involve multiple steps including computing geometric properties, specifying parameters manually, constructing and solving mesh size equations. The proposed method has a high automation level and requires no adjustments of parameters, such as spacing and weights, which are

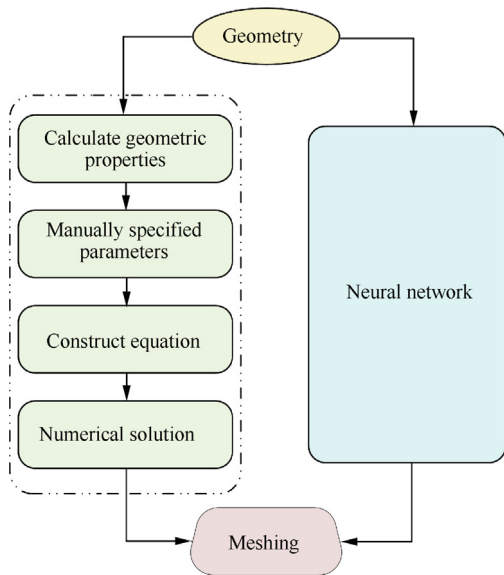


Fig. 3 Comparison between traditional method and proposed method using a flowchart.

highly dependent on human experience in traditional methods. Therefore, we propose IGNN, a method for geometrically adaptive mesh generation, which establishes a one-to-one mapping between implicit geometry representations (SDFs) and their corresponding MSFs. The proposed method includes a training phase and a testing phase, as illustrated in Fig. 4. The training set contains geometrical SDFs along with their corresponding MSFs. During the training phase, in order to optimize the model parameters, the generator takes in the SDF of the discretized Cartesian grid representation to generate an MSF, which is also a discretized Cartesian grid representation. The SDF serves as a conditional input for the discriminator, which also receives the generated and real MSFs to distinguish between them. In the testing phase, the geometrically adaptive MSF is derived by inputting the SDF corresponding to the new geometry into the trained generator. The generated MSF is input into the mesh generator to generate the corresponding unstructured mesh. By specifying the maximum and minimum element sizes of the global mesh cells for the generator, meshes of varying scales can be obtained.

3.1. Network architecture

In this section, we present the core of the proposed methodology, the generator module, and the discriminator module. Mathematically, the generator module constructs a mapping

$$f(x) = F(\phi; \theta) \quad (10)$$

where θ represents network parameters, ϕ and f represent the SDF and MSF respectively, and x denotes an arbitrary point within the domain to be meshed. The role of the discriminator is to optimize the quality of the MSF generated by the generator in a minimax game. In addition to the adversarial loss commonly used in general GANs, we have also incorporated a feature reconstruction loss to enhance the stability of adversarial training. In this section, we provide a comprehensive explanation of the theory and architecture underlying the proposed method.

3.1.1. Generator-discriminator module

Inspired by cGAN, we employ generators in our GAN framework to establish a one-to-one mapping between implicit geometry and MSF. The generator of cGAN incorporates two inputs, random noise z along with the conditioning variable y . The noise z represents the variability among samples within the same category, thus implying that the generation process can be perceived as a one-to-many mapping. However, our model aims to establish a one-to-one mapping from implicit geometry to MSF. Therefore, similar to the approach in pix2pix, we eliminate the noise z and solely utilize the condition y as input for the generator.

In our approach, both SDFs and MSFs are represented as matrices since they are discretized using a uniform Cartesian grid. The generator's task involves a matrix-to-matrix transformation, akin to the image-to-image translation problem, which is defined as the process of mapping a high-resolution input grid to a corresponding high-resolution output grid.⁴¹ Despite the varying values in the uniform grid, they possess a shared underlying structure, thereby ensuring alignment between the input and output structures.

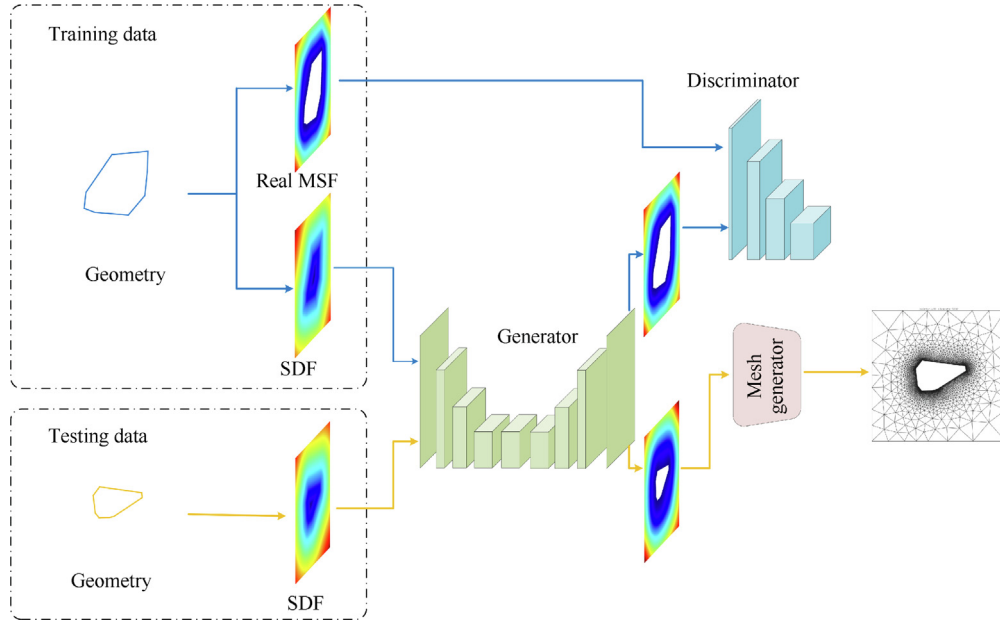


Fig. 4 Detailed flowchart of the proposed method (The blue lines indicate the data flow in training phase, while the yellow lines indicate the data flow in testing phase).

Previous approaches to image-to-image translation commonly employ encoder-decoder networks, in which the input image is progressively downsampled in the encoder to extract high-dimensional features, and subsequently upsampled in the decoder using a reversed process to generate the output. For the matrix-to-matrix translation task in this study, we also employ an encoder-decoder structure and incorporate a set of residual blocks between the encoder and the decoder. Multi-layer convolution in encoders and decoders captures local and global geometry features. To capture more global geometry features, multiple residual blocks form a middle layer to learn the encoded features for capturing global geometry features with complex structures.

The generator’s architecture is illustrated in Fig. 5, comprising three components, downsampling, residual block, and upsampling. A detailed configuration of the network architecture for the generator is provided in section 3.2. According to He et al.,⁴³ residual connections facilitate the learning of the identity function, which is crucial for matrix-to-matrix tasks due to the shared underlying structure between neural network inputs and outputs. In each of our residual blocks, there are two convolutional layers with an equal number of convolutional kernels on both layers. The architecture of the residual block is illustrated in Fig. 6. Additionally, the utilization of upsampling and downsampling is motivated by the consideration of receptive field sizes. Given the requirement to change a large number of values on the grid in matrix-to-matrix translation, it is advantageous for each value in the output matrix to have a wide effective acceptance field in the input.

In most GAN discriminator architectures, a fully connected network is added after convolution, as depicted by the purple block in Fig. 7. A true or false scalar is generated to provide a single prediction for the entire input to the discriminator. To capture high-frequency features, it is sufficient to restrict the attention of the discriminator to the structure of local patches. Therefore, we introduce the patch discriminator, which specif-

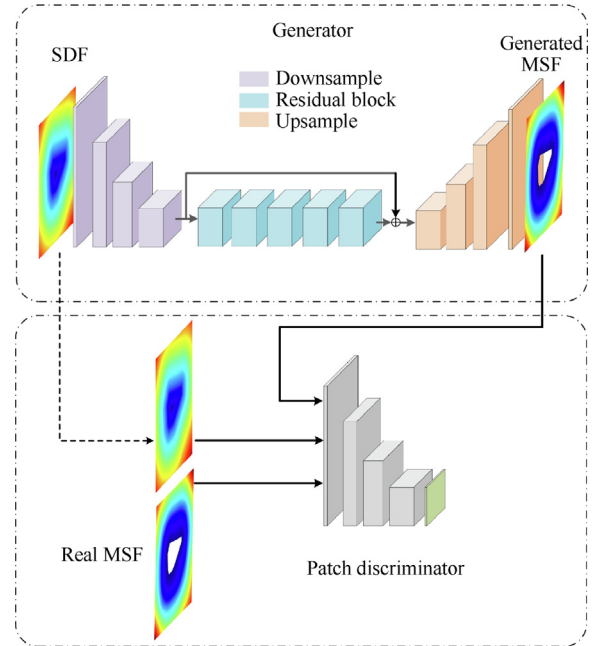


Fig. 5 Detailed architecture of IGNN, which consists of two modules: the generator and the patch discriminator.

ically discriminates at the scale of the structural block. The architecture of the patch discriminator is illustrated in Fig. 7. The patch discriminator outputs an $M \times M$ matrix rather than a simple true or false scalar. Each element in the matrix represents whether each $N \times N$ patch in the input is classified as real or fake by the discriminator. For instance, we fix the parameters of the discriminator structure at $N = 70$ (which remains constant), where each element in the $M \times M$ output matrix represents a 70×70 patch that corresponds to a portion of

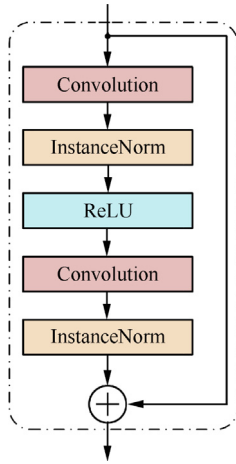


Fig. 6 Architecture of each residual block in IGNN.

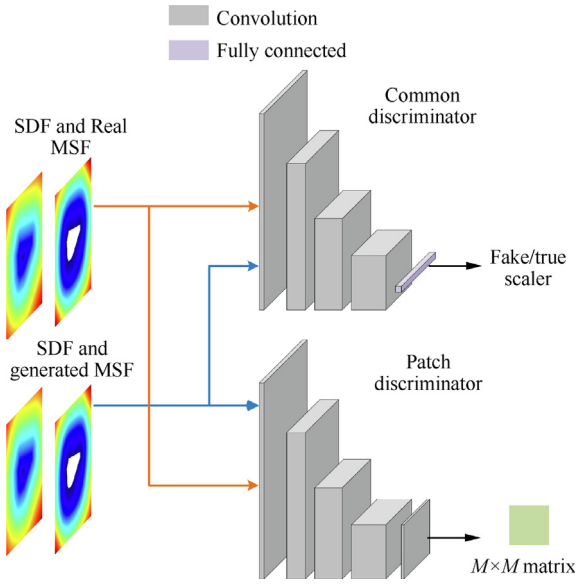


Fig. 7 Structural differences between patch discriminators and common discriminators (The yellow line represents the data flow of real data, while the blue line depicts the data flow of generated data).

the input rather than its entire size. It is important to note that the input is not simply divided equally into a specific number of 70×70 sub-blocks. The 70×70 patch represented by each element in the output matrix may overlap with the patches represented by other elements, thereby enabling the patch discriminator to capture high-frequency features beyond the element-wise level. Patch overlap in the discriminative output allows the patch discriminator to capture global geometry features well beyond the local scale. The patch Discriminator, in comparison to conventional discriminators, exhibits reduced parameters, accelerated discrimination and training processes, while effectively capturing significant high-frequency features.

3.1.2. Objective function

The proposed method involves two optimization objectives, namely adversarial loss and reconstruction loss. To ensure

stable training, we employ the adversarial loss defined in LSGAN, which replaces the cross-entropy loss used in traditional GANs with the least squares loss. The proposed model's adversarial loss is presented below,

$$L_{\text{adv}}(G, D) = \underset{G}{\text{argmin}} \max_D \left[E_{x,y} \left[(D(x, y))^2 \right] \right] + E_y \left[(1 - D(G(y), y))^2 \right] \quad (11)$$

where y represents the input condition from the distribution $p_{\text{data}}(y)$ of SDFs, x represents real data from the distribution $p_{\text{data}}(x)$ of MSFs, and $G(\cdot)$ and $D(\cdot)$ refer to the generator and discriminator respectively. The term “minmax” refers to the process of minimizing the least squares loss in G and maximizing it in D . The discriminator D must implicitly learn the similarity measure of the joint distribution of SDF and MSF.

Some previous GAN-based research has employed a combination of reconstruction loss and adversarial loss for guiding training. However, the use of element-wise measures (such as L1 or L2 loss) in reconstruction loss only captures low-frequency features and relies solely on the average of all outputs, which can lead to ambiguous results. Inspired by the content/style loss employed in previous studies on style transfer and image super-resolution,⁴⁴ we propose a novel reconstruction loss called Feature Reconstruction (FR) loss in order to effectively capture both low and high frequency features. Rather than encouraging the elements of the generated MSF to match the elements of the real MSF exactly, we encourage them to have similar feature representations so that both low and high frequency features can be captured. Specifically, we extract feature representations of both the generated MSF and the real MSF from multiple layers of the discriminator. The feature losses between the real MSF and the generated MSF in different layers are then measured, collectively constituting the FR loss. Let D_i represent the i th layer of the discriminator, and let $D_i(x)$ denote the feature representation extracted from input x at this layer. The FR loss is defined as the mean squared error between these feature representations:

$$L_{\text{FR}}(G, D) = \frac{1}{N} \sum_{i=1}^N \frac{1}{M_i} \left[\|D_i(x, y) - D_i(G(y), y)\|_2^2 \right] \quad (12)$$

where N represents the total number of layers, and M_i denotes the number of elements in the feature representation of each layer. The FR loss captures both low and high frequency features by producing statistics at different scales, thereby stabilizing the training process as the generator is required to conform to these statistics across multiple scales. Similarly, the FR loss calculates the feature similarity between the generated MSF and the real MSF at different scales, thus facilitating IGNN to capture geometry features at different scales, i.e., local and global geometry features. The final objective function of IGNN is therefore composed of two parts, as follows

$$L = L_{\text{adv}}(G, D) + \lambda L_{\text{FR}}(G, D) \quad (13)$$

where λ determines the relative importance of the two losses.

3.2. Configuration of neural networks

Both the generator and discriminator employ the complete CNN structure of DCGAN. Fig. 5 illustrates that the down-sampling and up-sampling processes in the network exhibit

symmetrical structures. Each downsampling block in the former includes Convolution, InstanceNorm, and Relu, while each upsampling block in the latter consists of Deconvolution, InstanceNorm, and Relu. Furthermore, each residual block contains two identical convolutional layers, specifically arranged in the following order, Convolution, InstanceNorm, Relu, Convolution, and InstanceNorm. In addition to the residual connections within each residual block, we also incorporate a skip connection between the inputs and outputs of the entire residual network. Table 1 presents more detailed generator network architectures, where normalization layers are utilized in all layers except the output layer. The discriminator contains five convolutional layers, with the exception of the output layer where each convolution is followed by InstanceNorm and LeakyReLU. The detailed structure of the discriminator network is presented in Table 2.

Under the strategy of using SDF as an implicit representation of geometry, the generator takes a 512×512 matrix as input and produces an MSF with the same dimensions. The training MSF is constructed through aforementioned Poisson’s equations on a uniform background grid and presented in matrix form. The discriminator takes both the generated MSF and the real MSF as input, with the SDF matrix serving as a condition. The generated discrete and normalized MSF enables the mesh generator to produce unstructured meshes of varying scales by adjusting the global maximum and minimum sizes of mesh elements.

4. Experimental results

4.1. Data preparation

We designed a series of experiments for demonstrating the capability of IGNN for geometrically adaptive mesh generation. Generating training data with input–output pairs is an important requirement for proper training. The dataset is generated through a pipeline that consists of the random geometry generation and the corresponding SDF and MSF calculations. We use the generated data to demonstrate that the proposed method is able to learn the underlying relationship between the geometry and the corresponding MSF.

4.1.1. Dataset

The basis for creating the dataset involves generating random geometry. The pipeline used for this purpose is illustrated in Fig. 8, where we categorize the randomly generated geometries into two groups, polygonal and curvilinear shapes. To generate the random geometry, we divided the pipeline into modules, each represented by a yellow circle.

The first step in creating geometry is to sample random points. During this stage, a diverse set of points is collected within a restricted area, with the restriction being a rectangular region. The second step involves creating the convex hull. Once the point set has been sampled, we proceed to construct the convex hull from that set of points. The process of generating random polygons using the hull of the point set is illustrated in Fig. 9. The third step involves determining the type of geometry to be created, which can be categorized primarily into polygonal and curvilinear shapes. The fourth step in geometry generation involves generating polygonal or curved shapes. To generate polygons, the convex hull constructed in the second step can be directly used to form them. For curvilinear shape, a spline is created by randomly placing midpoints between the vertices of the convex hull. The data samples consist of different shapes, which are generally composed of line segments, corner points, and curve segments, and have global consistency despite their local differences. Especially because the adoption of SDF as a unified implicit representation of geometry and the design of FR loss and patch discriminator in IGNN make it possible to capture more global features.

After generating the random geometry, the next step involves constructing the input–output paired training data, the SDF and MSF corresponding to the random geometry. To generate the inputs for our model, we utilized FMM³⁶ on a 512×512 uniform grid to obtain the SDF. Additionally, we employed fenics⁴⁵, an open-source finite element framework, to solve Poisson’s equations with specified nodal/linear sources on a 512×512 uniform mesh and acquire corresponding MSF. The configuration for solving the Poisson equation with specified sources for polygonal geometries is depicted in Fig. 10, where the nodal sources are represented by blue points and the linear sources are denoted by yellow lines. For curvilinear shape geometries, where the arrangement of sources

Table 1 Detailed configurations of architectures for generator.

Layer	Kernel	Stride	Normalization	Activation
Input	7	1	InstanceNorm	Relu
DownSample1	3	2	InstanceNorm	Relu
DownSample2	3	2	InstanceNorm	Relu
DownSample3	3	2	InstanceNorm	Relu
DownSample4	3	2	InstanceNorm	Relu
Residual $\times 11$	3	2	InstanceNorm	Relu
	3	2	InstanceNorm	
UpSample1	3	2	InstanceNorm	Relu
UpSample2	3	2	InstanceNorm	Relu
UpSample3	3	2	InstanceNorm	Relu
UpSample4	3	2	InstanceNorm	Relu
Output	7	1	InstanceNorm	Sigmoid

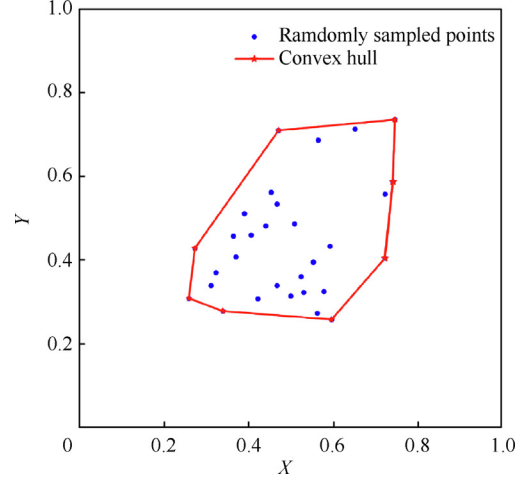
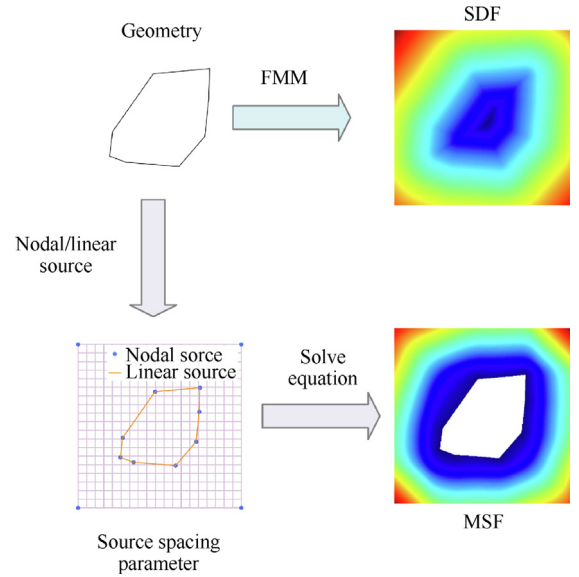
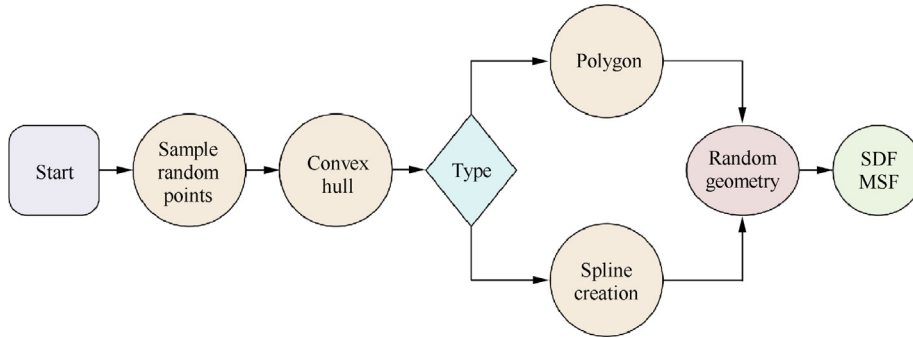
Table 2 Detailed configurations of architectures for patch discriminator.

Layer	Kernel	Stride	Normalization	Activation
Input	4	2	InstanceNorm	LeakyReLU
1st Conv	4	2	InstanceNorm	LeakyReLU
2nd Conv	4	2	InstanceNorm	LeakyReLU
3rd Conv	4	2	InstanceNorm	LeakyReLU
Output	4	1		

can be viewed as the positioning of multiple consecutive point sources. It should be noted that the uniform background grid is solely for illustrative purposes, not representing the actual number of grids. Upon specifying the appropriate spacing parameters at the sources, one can obtain the corresponding MSF on the uniform grid. The process of constructing input-output paired data is shown in Fig. 10. The entire dataset consists of 2502 samples, which is divided randomly into training and test sets, with a ratio of 85% and 15%, respectively.

To further validate the practical significance of IGNN and its generalization to different geometries, we use 2D cylinder, NACA0012, and RAE2822 in CFD as validation. In particular, the 2D cylinder is validated on the model with polygon and spline geometry as the training dataset. NACA0012 and RAE2822 are validated on the model with wing data as the training dataset. Using the RAE2822 airfoil as the baseline, we use 12 class shape transformation parameters for geometry parameterization and construct an airfoil dataset by randomly selecting 1800 airfoils within the plus or minus 30% perturbation interval through Latin hypercube sampling. The sampling results of the airfoil dataset are shown in Fig. 11, where the training set and test set have a proportion of 85%, and 15% respectively.

Sample data preprocessing is crucial for neural network learning because data scale uniformity and location consistency can facilitate IGNN processing of geometry conditions with different sizes and locations. We perform scale normalization and geometry centering on the samples, specifically normalizing the MSF to the $[0,1]$ range and scaling the geometry to the center of the 1×1 square region with $(0.5,0.5)$ as the center. Normalization and centering allow IGNN to learn more about the impact of geometry features

**Fig. 9** Creation of stochastic simple geometries by taking the convex hull (red lines) of randomly sampled point sets (blue points).**Fig. 10** Process of training and testing data preparation.**Fig. 8** Pipeline for random geometry generation.

on MSF without having to consider different scales and locations of samples. In addition, for some computational domains that are much larger compared to the geometry, the computation focuses on the domains close to the geometry. Therefore, a reasonable mesh distribution can be obtained simply by obtaining the mesh sizes of the computational domains around the geometry, since the domains far away from the geometry are extremely coarse meshed. In this way, we further extend IGNN to computational domains of different sizes and locations.

4.1.2. Implementation and configuration

The training process of IGNN is shown in Fig. 12 where the geometry is preprocessed to obtain the implicit geometry SDF and the mesh size MSF (circle), respectively. The SDF is used as an input to the generator to generate the MSF (square), and the discriminator determines the generated and real MSFs to get the loss 1, i.e., the adversarial loss. The feature similarity error between the generated and real MSFs is computed separately at different layers of the discriminator to obtain Loss 2, the feature reconstruction loss. The weighted sum of Loss 1 and Loss 2 is used to optimize the generator and discriminator parameters.

The neural network is trained and tested in Python using the open-source Deep Learning framework Pytorch. To optimize the parameters of the generator and discriminator, we employ two ADAM optimizers for parameter optimization respectively. Specifically, we initially set the learning rate of

the Adam optimizer to 0.001 for training stochastic gradient descent on the model parameters and subsequently decayed the learning rate by a factor of 0.99 per epoch. To enhance model convergence and minimize errors during training, the dataset is normalized to a range of 0–1. However, there is no need to normalize the output data of the generator as it already falls within the 0–1 range.

After generating the MSF, the final step is to automate the meshing process. We use gmsh⁴⁶ to generate the mesh directly from MSF. This paper primarily focuses on the varying spatial resolutions around random geometries, thus fixing meshes within a square domain. For the same mesh generator, the input of a high-quality MSF consistently yields a superior mesh compared to a low-quality MSF. Consequently, for the quantitative assessment of mesh quality, we adopt the quantitative evaluation metrics of MSF. The output of the neural network is a matrix, thus the Mean Absolute Error (MAE) metric is employed to quantitatively assess the model's performance:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |x_i - \hat{x}_i| \quad (14)$$

where x and $\hat{x}$ represent the predicted results and real data, respectively, and n denotes the number of elements. All experimental examples were conducted on a machine equipped with an Intel I9-10900K CPU and Nvidia Quadro RTX5000 GPU.

4.2. Results and discussion

The performance of IGNN is influenced by various components of the model, including the weight assigned to each loss component, the choice of the loss function, the structure of both discriminator and generator networks, among others. Therefore, it is crucial to investigate and compare these components to select an optimal network that can generalize well in practical usage. The following sections first compare and contrast the various choices of these model components, and then examine the predictive effectiveness of IGNN under different geometries as well as its overall validation

4.2.1. Influence of hyper-parameter

The weight hyperparameter λ is a scalar that balances the adversarial loss and reconstruction loss in Eq. (14). To demonstrate the impact of λ , we quantitatively evaluate the model performance using the MAE metric under different λ settings.

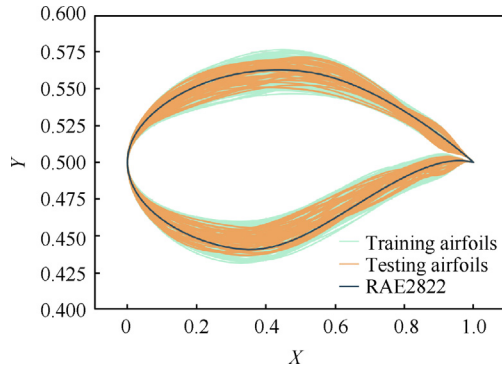


Fig. 11 Illustration of the training set (green) and test set (yellow) for airfoil samples.

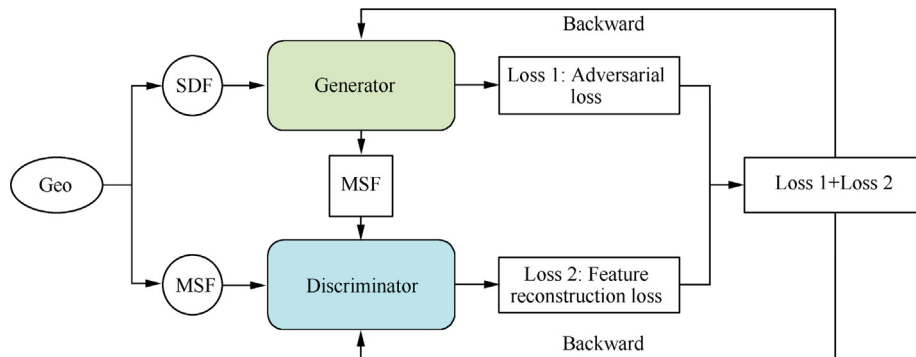


Fig. 12 Detailed flowchart of the training process of the proposed method.

We select λ values ranging from 1 to 45 and assessed their impact on performance by calculating the average of MAE loss of the predicted results for each λ setting. Fig. 13 illustrates the performance metrics for different λ values, and it can be observed that the model error decreases as λ increases from 1 to 25, but then starts to rise again for λ greater than 25. Moreover, for λ values greater than 20, there is no significant change in the error values, indicating that the results obtained from these λ are not significantly different. Therefore, for the following study, we chose to use $\lambda = 25$ to strike a balance between adversarial loss and reconstruction loss.

4.2.2. Choice of loss function

We have mentioned that solely considering the adversarial loss in model training leads to generated results being independent of the original ones. To establish a one-to-one mapping between geometry and MSF, it is necessary to incorporate the reconstruction loss into the total loss. Consequently, the generator's loss comprises two components, the adversarial loss and the reconstruction loss. While adversarial loss plays a crucial role in GANs, our focus here lies on selecting the appropriate reconstruction loss. In this comparison, we evaluate the FR loss along with L1 and L2 losses. Randomly selected samples from the test set were utilized to assess the efficacy of various reconstruction losses in evaluating the fitting ability of IGNN.

Fig. 14 presents a boxplot illustrating the mean values of MAE corresponding to various loss function settings. The low mean (green triangles) and median (yellow line) MAE values associated with FR loss demonstrate a significant enhancement in prediction accuracy. The smoothness of the L2 loss in the parameter space leads to results that converge towards the mean. In contrast, the L1 loss produces obviously higher accuracy compared to the L2 loss. Additionally, the FR loss, due to its consideration of multi-scale statistics, yields the highest prediction accuracy compared to both L1 and L2. The reference MSF and the corresponding predictions from L1, L2, and FR losses for four randomly selected test cases are depicted in Fig. 15. The corresponding errors of the results from various loss settings are illustrated in Fig. 16, where a clear difference can be observed, even though these predictions in Fig. 15 appear similar. The MSF predicted using L2 loss has the highest overall error compared to the reference MSF, followed by L1 loss which has a slightly lower error, and FR loss which has

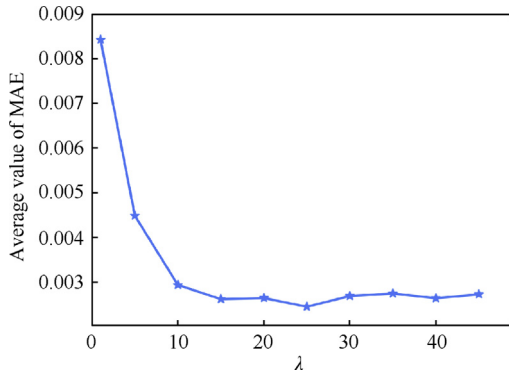


Fig. 13 Mean value of MAE loss in prediction results under different λ values.

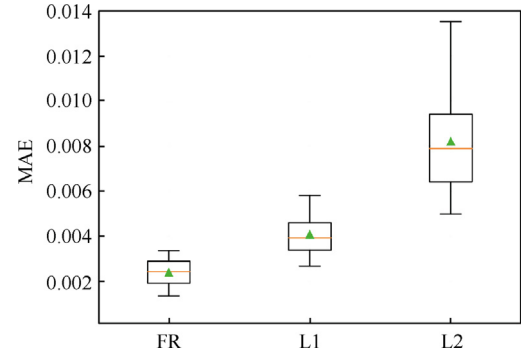


Fig. 14 Boxplots of MAE for the test set across different reconstruction loss settings.

the lowest error. The lower errors indicate that the MSFs generated using the IGNN with FR loss are accurately predicted at both local and global geometry features. Due to variations in the accuracy of predicted MSF caused by different losses, there is a significant difference in the final generated mesh. The comparison between the meshes predicted using different losses and the reference meshes for the four test cases is illustrated in Fig. 17. The meshes obtained by the L1/L2 loss are less accurate in terms of element distribution in regions with prominent geometric features compared to the reference meshes, while the results with FR loss provide more accurate element distribution throughout the computational domain. L1/L2 loss as an element-wise measure captures only low-frequency features. In contrast, the method with FR loss extracts features from multiple layers of the discriminator and is able to capture both low-frequency and high-frequency feature generation, thus generating more accurate results. Compared to the L1/L2 loss, it is evident that the FR-loss model generates a mesh that exhibits greater consistency with the reference mesh in terms of spatial resolution distribution at critical regions.

4.2.3. The influence of the patch discriminator

In this section, we compare the structures of discriminator networks using a patch discriminator and those using a common discriminator to demonstrate the superiority of the patch discriminator. The network structure of the common discriminator is built upon that of the patch discriminator. The structure of the patch discriminator can be found in Table 2, whereas Table 3 illustrates the structure of the common discriminator. Except for changing the patch discriminator to the common discriminator, all other experiment settings remain unchanged. Particularly, the FR loss adopted in these models utilizes the first four layers of features from the discriminator. As demonstrated in Table 2 and Table 3, identical settings are employed for both the first four layers of the patch discriminator and the common discriminator, ensuring the fairness of the comparison.

In Fig. 18, the convergence curves of the generator, discriminator, and MAE losses are displayed for different discriminator settings. The use of a patch discriminator results in a lower loss for the generator compared to that with no patch discriminator setting (i.e., the common discriminator). Fig. 19 displays the average MAE values of four random samples in the test set, comparing the results with and without utilizing the patch

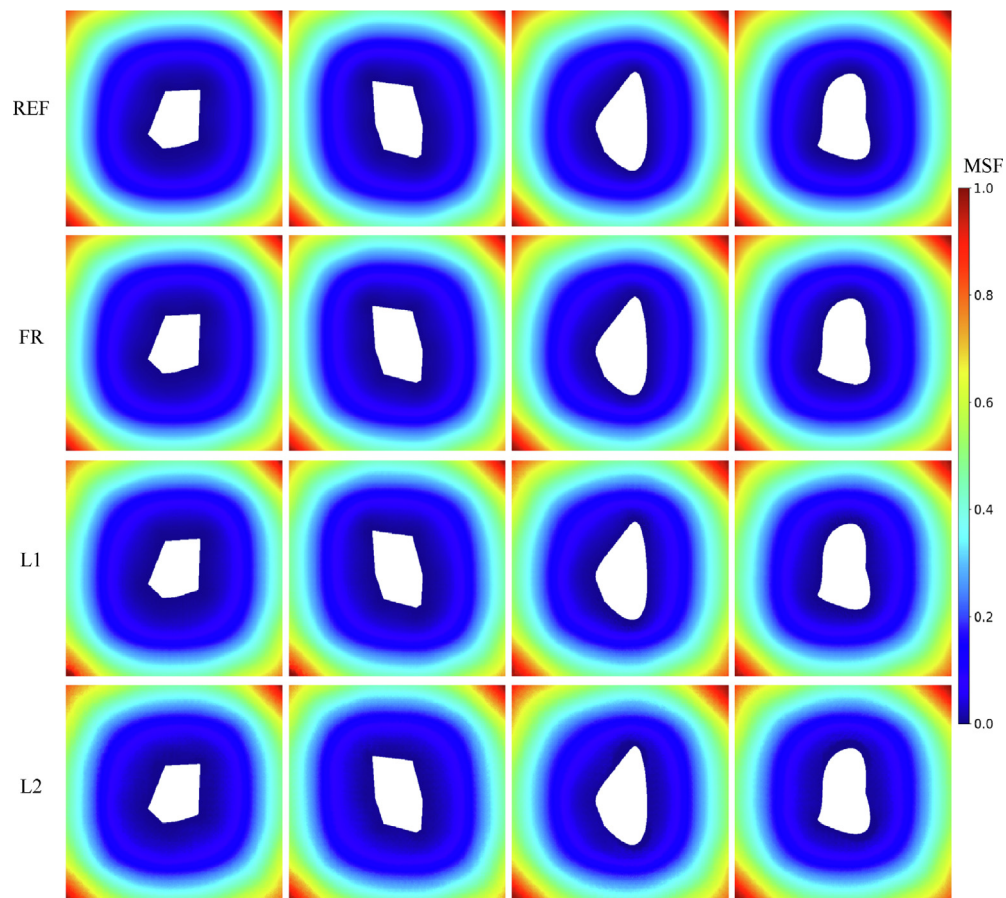


Fig. 15 Comparison of MSFs predicted with different loss functions for four randomly selected test geometries.

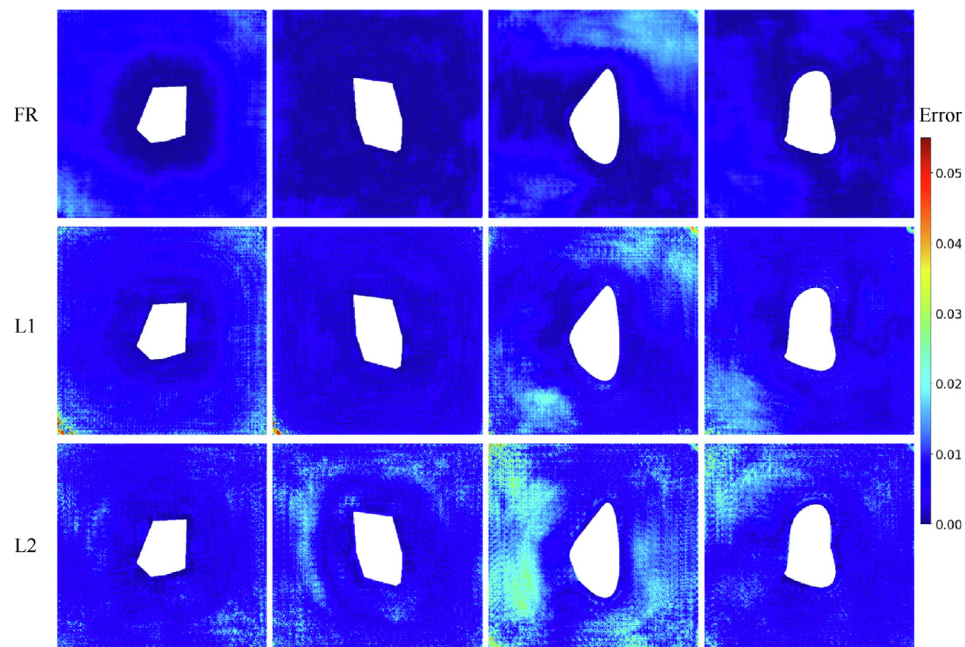


Fig. 16 Errors between the predicted MSF and the reference MSF are compared for four randomly selected test geometries, considering different choices of loss functions.

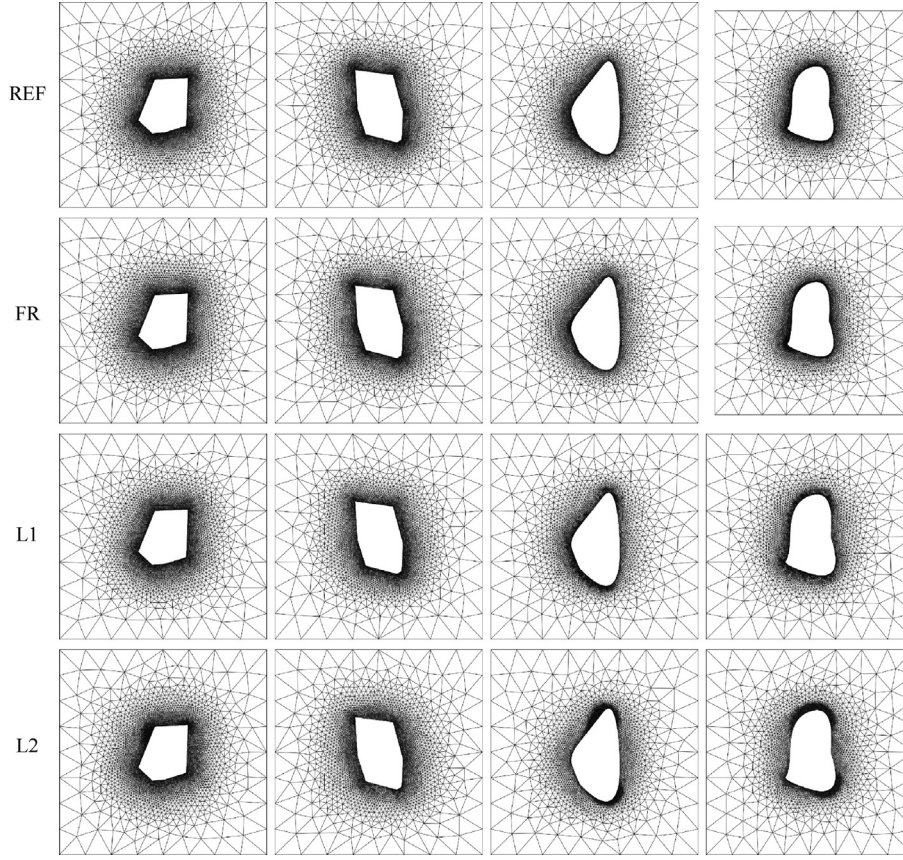


Fig. 17 Comparison between reference meshes and predicted meshes generated using different loss functions for four randomly selected test geometries.

Table 3 Detailed configurations of the architectures for the common discriminator.

Layer	Kernel	Stride	Normalization	Activation
Input	4	2	InstanceNorm	LeakyReLU
1st Conv	4	2	InstanceNorm	LeakyReLU
2nd Conv	4	2	InstanceNorm	LeakyReLU
3rd Conv	4	2	InstanceNorm	LeakyReLU
4th Conv	4	2	InstanceNorm	LeakyReLU
5th Conv	4	2	InstanceNorm	LeakyReLU
6th Conv	4	2	InstanceNorm	LeakyReLU
Output	4	3		

discriminator. While the common discriminator outputs a scalar to measure the truth of the entire input, the patch discriminator produces an $M \times M$ output matrix where each element represents a measure of a patch. Consequently, the patch discriminator enables more precise evaluations of inputs compared to a common discriminator, resulting in a reduced MAE. The predicted MSFs and errors, with and without the utilization of the patch discriminator, are presented in Fig. 20 for the four test geometries. Compared to the reference MSFs, the MSFs predicted using the patch discriminator (sec-

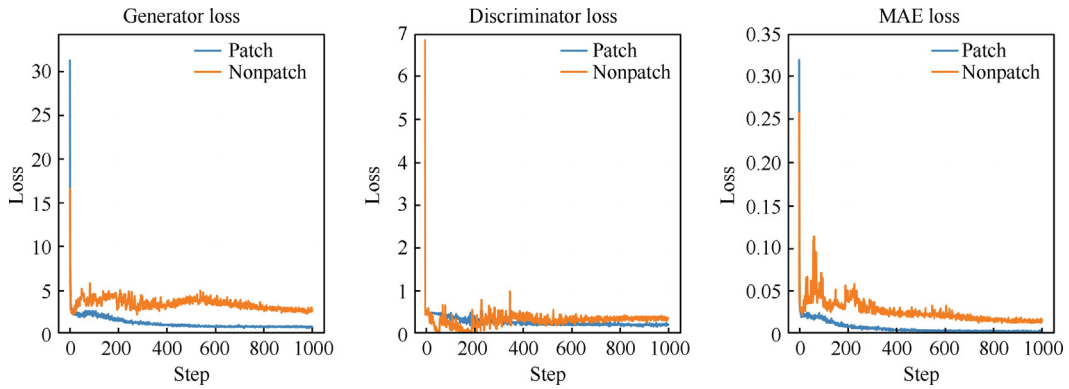


Fig. 18 Convergence curves of the generator loss, discriminator loss, and MAE loss with and without the patch discriminator.

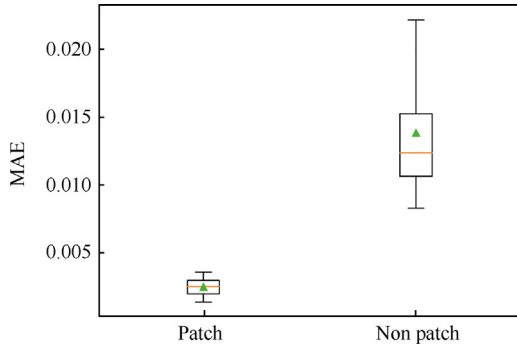


Fig. 19 Average MAE value for test samples.

ond row) are significantly smoother than those without the patch discriminator (third row).

The fourth and fifth rows (prediction error) demonstrate that the introduction of the patch discriminator effectively

reduces errors in the prediction results. The low global error of the results generated by IGNN with the patch discriminator illustrates that the patch discriminator efficiently captures global geometry features. Fig. 21 illustrates the reference mesh, as well as the meshes obtained with and without the patch discriminator. It can be observed that the mesh generated without the patch discriminator exhibits significant errors in element distribution near the geometry, whereas the mesh generated with the patch discriminator demonstrates a more precise distribution of elements. These comparisons show the patch discriminator enables judgments at finer scales, resulting in more accurate results. Consequently, the mesh obtained by using the patch discriminator exhibits superior spatial resolution variation.

4.2.4. Choice of network structure

In the proposed method, the generator incorporates both upsampling/downsampling blocks and a varying number of residual blocks. This section aims to investigate the impact

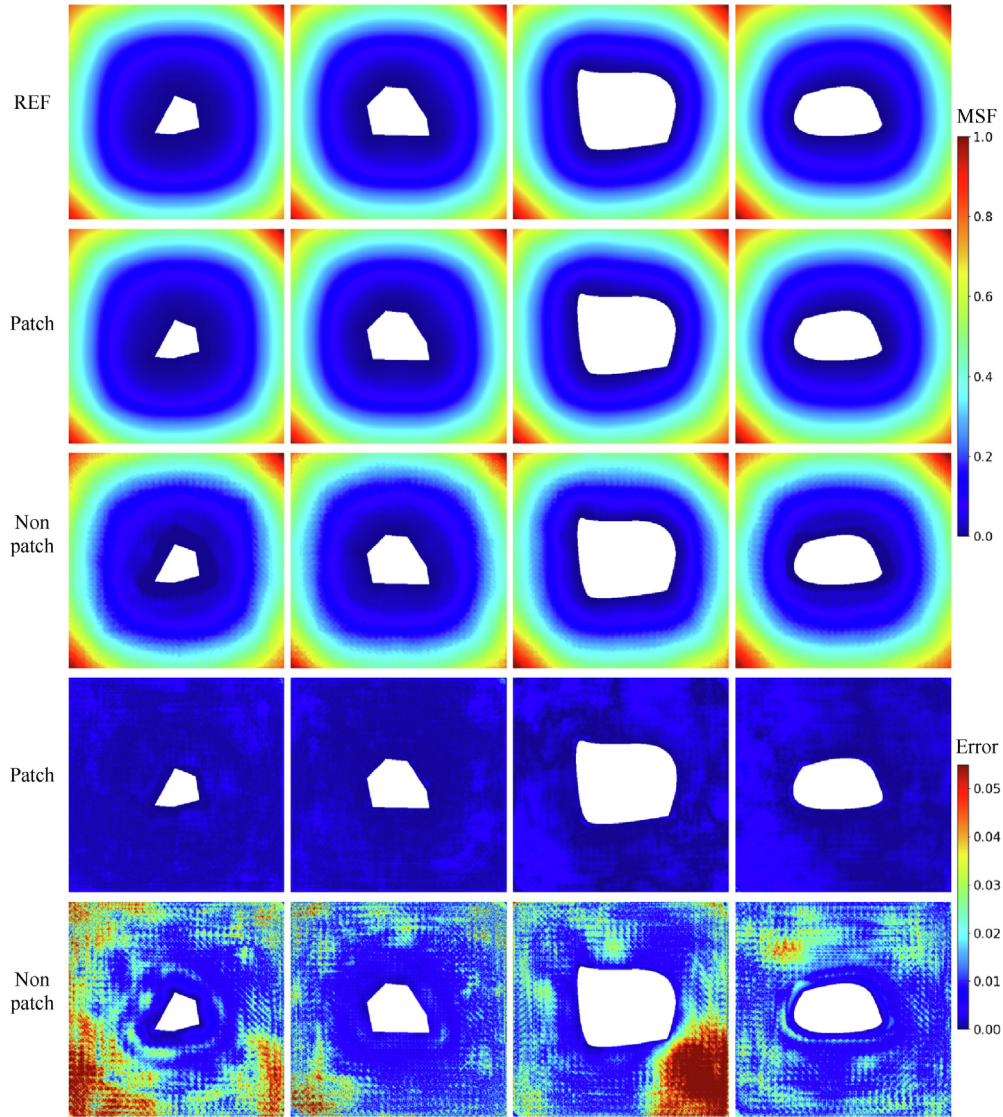


Fig. 20 Comparison of MSFs predicted for the four randomly selected test examples from models with and without utilizing the patch discriminator.

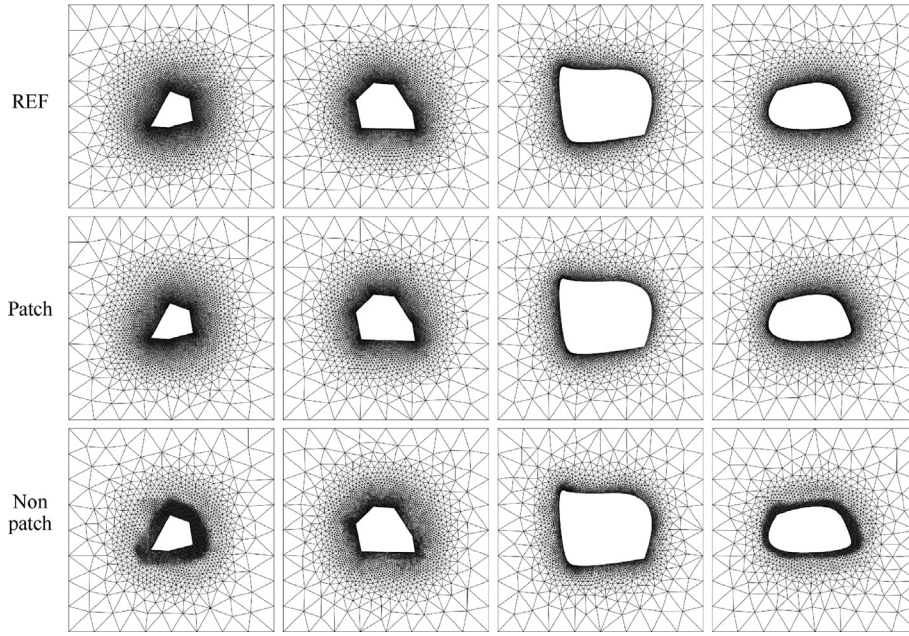


Fig. 21 Comparison between reference meshes and predicted meshes for the four randomly selected test examples from models with and without the patch discriminator.

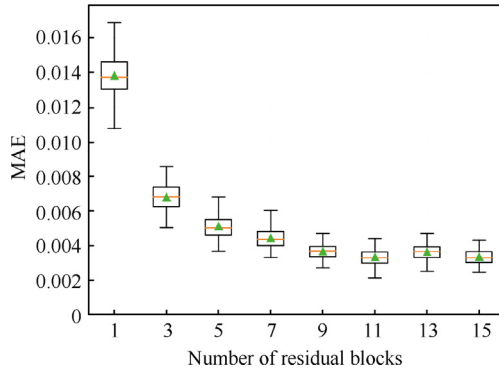


Fig. 22 Boxplot of the MAE of the prediction results when the number of residual blocks varies from 1 to 15, with the medians displayed in orange lines and the means in green triangles.

of different numbers of residual blocks on the predictive performance of the generator, ranging from 1 to 15 in our experiments. Fig. 22 illustrates the corresponding MAE values for the test set as the number of residual blocks increases from 1 to 15. It can be observed that the model error gradually decreases as the number of residual blocks increases from 1 to 11, but starts to increase when it exceeds 11. Thus, it is evident that the mean error value (green triangles) reaches its minimum with 11 residual blocks. Since increasing the number of residual blocks cannot further improve the model’s accuracy, 11 is an optimal choice due to its significantly lower computational cost.

4.2.5. Overall validation of IGNN

To verify the generalization of IGNN to different geometries, we particularly assess the model’s capability to accurately pre-

dict MSFs using four randomly selected test geometries on a 512×512 scale. We randomly select two polygons and two curve shapes from the test set. The trained model is used to predict the corresponding MSFs displayed in Fig. 23, as well as the reference MSF and the error between them. It can be seen that the model effectively captures the prominent features of the geometric periphery, leading to varying mesh spatial resolutions in these critical regions. Since the MSF is normalized to a range of 0–1, meshes of different scales can be generated by specifying different maximum and minimum element sizes in the generator, which are the only two parameters that need to be assigned in advance.

In Fig. 24 the meshes obtained from the model predictions and reference are shown for the four random test geometries. By specifying larger element sizes with a minimum of 0.0035 and a maximum of 0.35, coarse meshes are generated and shown in the second row along with the reference mesh shown in the first row. By specifying smaller element sizes with a minimum of 0.001 and a maximum of 0.3, the fine meshes are generated and shown in the fourth row along with the reference mesh displayed in the third row. It can be seen from these plots that the generated coarse and fine meshes are almost identically distributed with the references as a whole. The zoomed-in views of fine meshes for the critical local regions are presented in the last two rows. Similarities in the element distribution and spatial resolution are also observed between the references and the generated meshes. Comparison experiments here indicate that the high accuracy in the predicted MSFs ensures the accuracy of the generated meshes in the whole domain and on various scales. In addition, for fine mesh generation with spline geometry, the parameterization method of the spline with a nonuniform arrangement of control points can lead to large local feature variations in areas where the curve appears flat overall. Because IGNN accurately captures

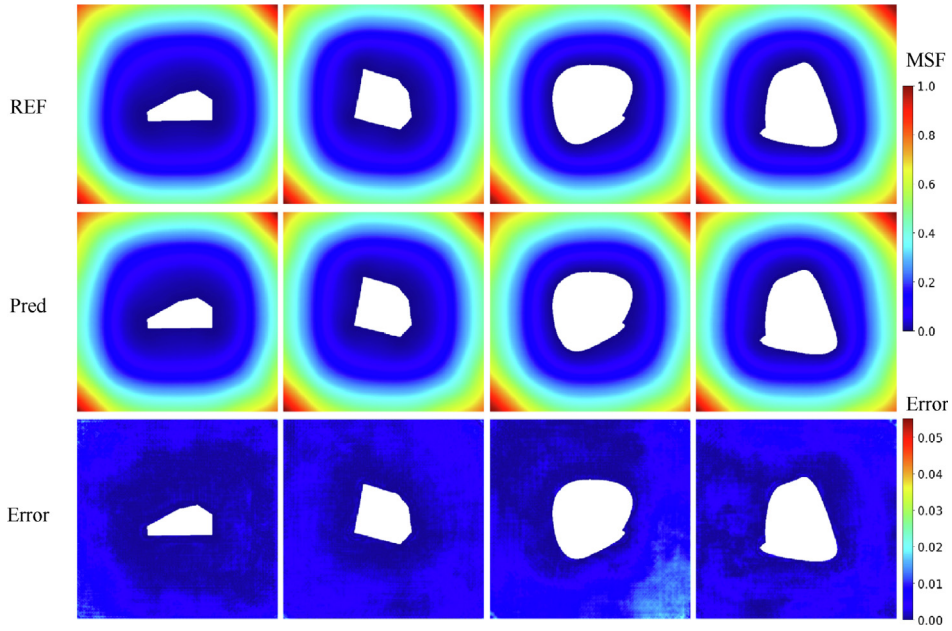


Fig. 23 Comparison between the predicted and the reference MSFs for four randomly selected geometries in the test dataset.

local features, these larger local feature variations are captured by IGNN, which generates denser mesh cells in that area to ensure an accurate representation of geometric details.

4.2.6. Application of IGNN on real cases

To further validate the practical significance of IGNN and its generalization to real-world geometries, we use 2D cylinder, NACA0012, and RAE2822 as validation. The predicted and reference MSFs and errors for 2D cylinder, NACA0012, and RAE2822 are shown in Fig. 25. The low error of 2D cylinders indicates that IGNN shows better generalization on polygonal and spline geometries as the training dataset. For NACA0012 and RAE2822, RAE2822 has a very low error, and although the error for NACA0012 is slightly higher compared to RAE2822, it also has a low prediction error, which correlates with the wing dataset being based on RAE2822. The large difference between NACA0012 and the training set based on RAE2822 also demonstrates the superior generalization of IGNN over the airfoil.

The predicted and reference meshes for the 2D cylinder, NACA0012, RAE2822, are shown in Fig. 26. Specifically, the cylinder shape is significantly different from the training data, especially in terms of curvature and symmetry. The comparison of the training geometry with the 2D cylinder shows that IGNN is still able to generate high-quality meshes when dealing with a geometry that is significantly different from the training data. The predicted meshes of NACA0012 and RAE2822 exhibit higher resolution in the leading and trailing edge areas of the airfoil, where the curvature is greater and physical volume gradients are larger, so finer meshes are needed to capture flow details. In the middle area of the airfoil, the low curvature areas are discretized at a lower resolution, and the overall mesh distribution is homogeneous, which ensures the accuracy and efficiency of the flow field calculation. Furthermore, even though the training set is based on a pertur-

bation of RAE2822, and NACA0012 as a symmetric airfoil that is significantly different from the training data, the generated mesh still conforms well to the geometry features. This shows that IGNN has practical significance in having better generalization and robustness and can be applied to shapes in real-world practice.

4.2.7. Efficiency validation of IGNN

We have verified the superior performance of each optimal choice for parameters and modules in the model in the previous subsections. In this subsection, we first demonstrate the overall performance of the optimal model by presenting the convergence process and prediction accuracy in both the training and test sets. We next validate the effectiveness of the model in generating meshes on various scales for individual test examples.

The convergence curves of the model on the training and test sets are shown in Fig. 27, represented by the blue and yellow lines respectively. The curves visually demonstrate the iterative process and indicate that the training loss levels off towards the end. To further quantitatively analyze the accuracy of the model, the boxplot of MAEs is presented in Fig. 28 for all samples in both the training and test sets. The lower mean (green triangle) and median (yellow line) of the training set losses suggest the superior capacity of model fitting during the training phase. It is common for the test loss to obtain higher loss values and MAEs compared with those of training data due to the absence of model exposure to test samples during training, indicating the trained model does not overfit the training data. Although the test data have larger overall MAEs compared to those of the training samples, the predicted MSFs and the corresponding generated meshes for four randomly selected test examples shown in Fig. 23 and Fig. 24 respectively, demonstrate that there is no significant difference between the generated MSF and reference MSF.

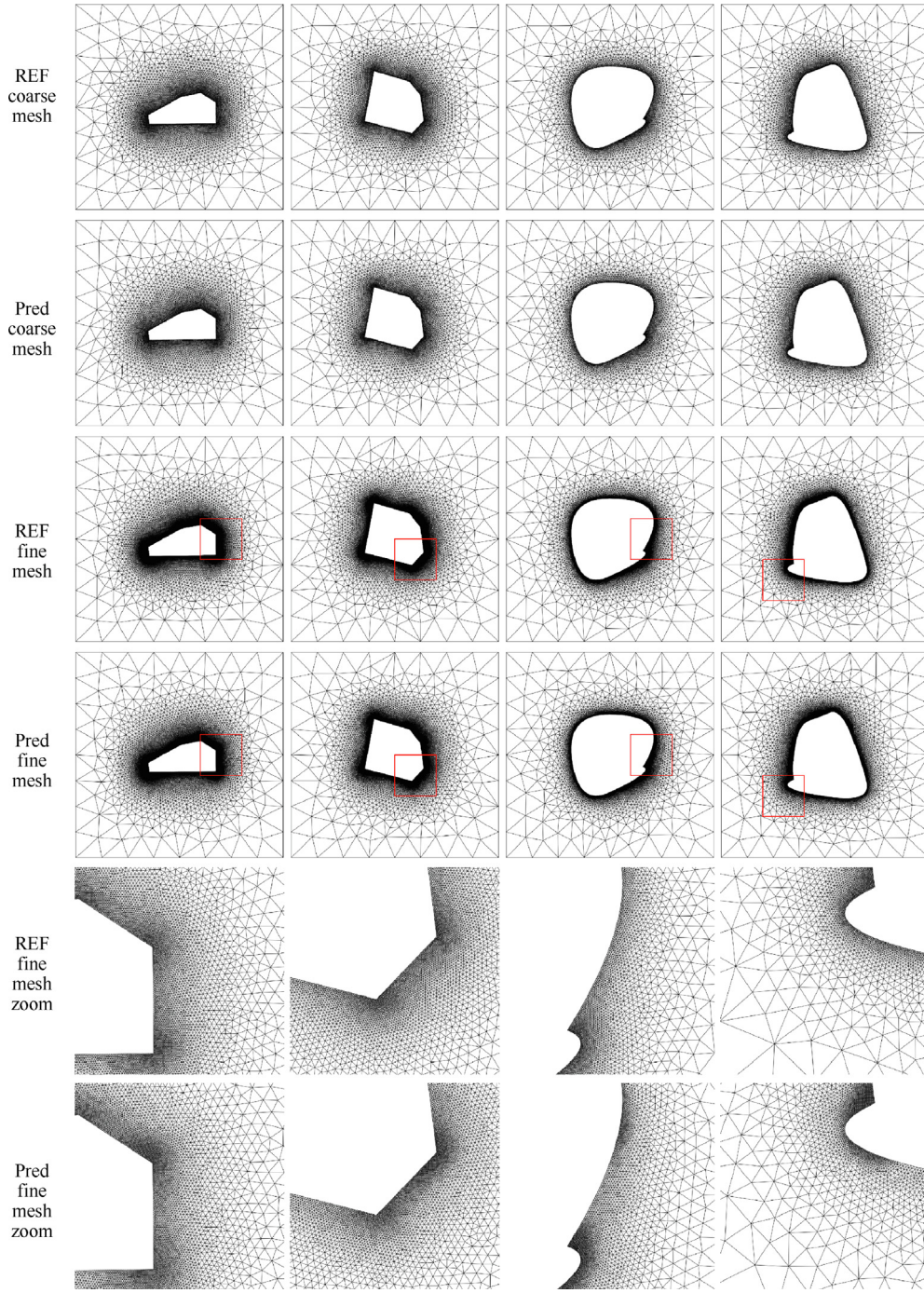


Fig. 24 Predicted and reference meshes correspond to four randomly selected geometries in the test dataset (Coarse and fine meshes, each with different scales, are presented separately).

Hence, it can be inferred that IGNN has the capability to accurately predict the MSF corresponding to random geometry within a given geometric space.

In terms of the efficiency of IGNN, it was trained with a batch size of 16 and took an average of 10.257 h. Although the training process takes a little longer, this is because the model needs to optimize the data to learn the complex mapping relationship between geometry and MSF, and the training of the model is a one-time deal. We compare the time consumption and resource usage of IGNN for mesh generation,

respectively. Table 4 shows the average runtime and memory usage of the proposed method and the traditional method on the test set. Since the neural network is trained on the GPU, the runtime of IGNN is over two orders of magnitude faster than the traditional method when tested on the GPU. Considering that the traditional method is solved on the CPU, we also tested the average running time of IGNN on the CPU. Considering that the traditional methods are solved on the CPU, we also tested the average runtime of IGNN on the CPU, which is more than an order of magnitude faster than the traditional

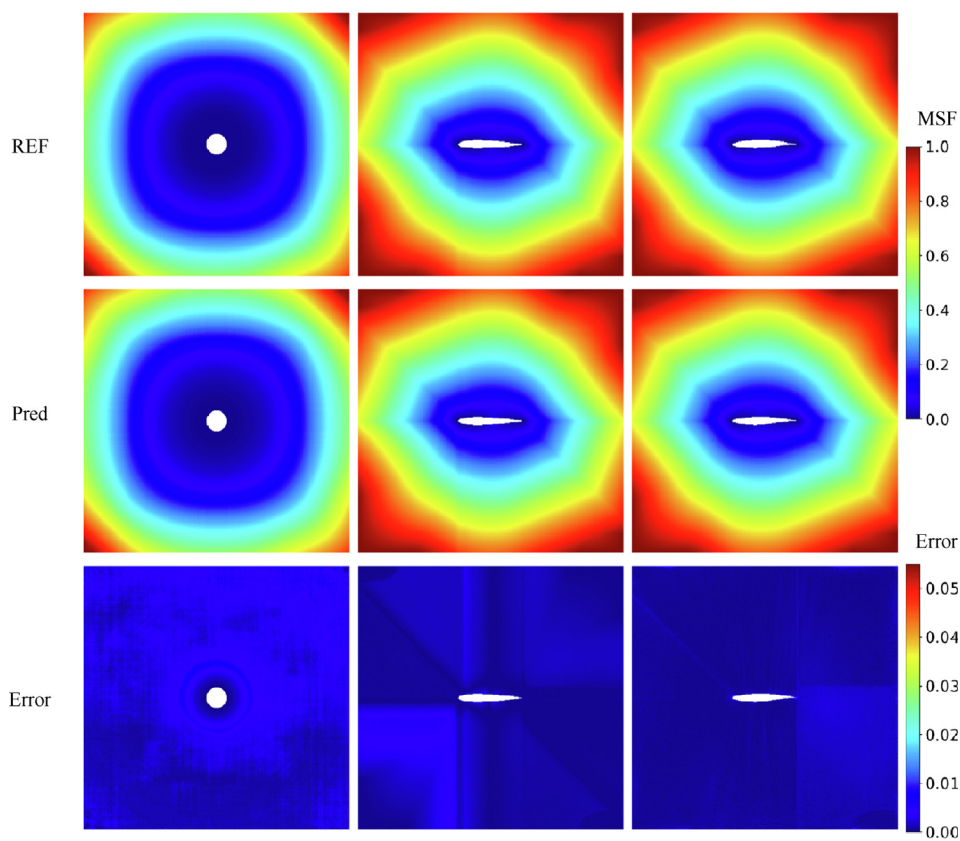


Fig. 25 Comparison of predicted MSF with reference MSF for 2D cylinder, NACA0012 and RAE2822.

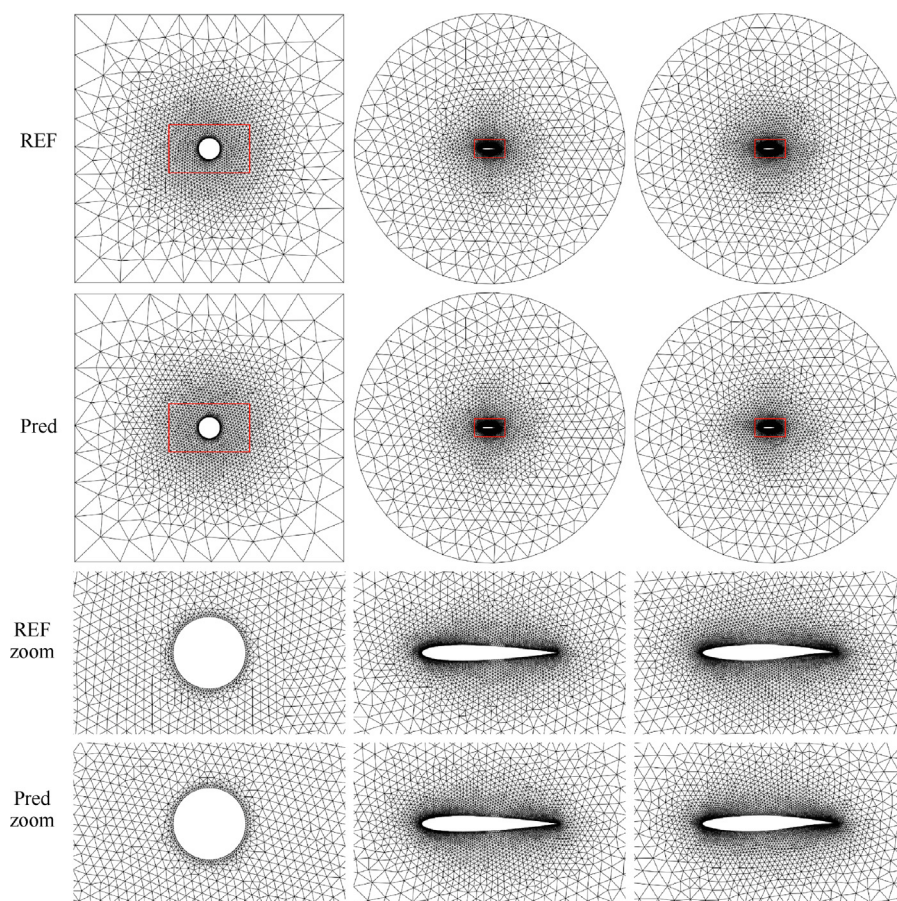


Fig. 26 Comparison of the predicted and reference meshes corresponding to the 2D cylinder, NACA0012, and RAE2822.

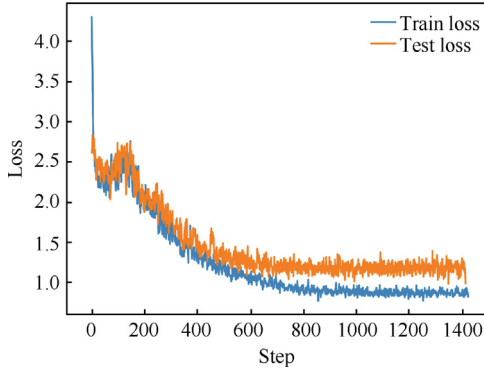


Fig. 27 Convergence curves of the losses on the training set and test set.

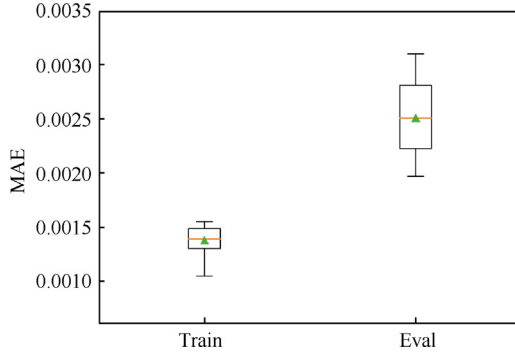


Fig. 28 Boxplots of the MAEs for the training and test sets.

Table 4 Average running time and memory usage of the proposed method and traditional method on the testing set.

Method	IGNN (GPU)	IGNN (CPU)	Traditional
Time (s)	0.109	0.882	30.173
Memory (MB)	1432.5	1310.7	1105.9

methods. It is important to note that the runtime of the traditional method does not include the time spent on the manual parameter adjustment and optimization process. IGNN uses 1432.5 M of memory when running on the GPU, which is 29.5% more than the traditional method but takes only 0.36% of the time of the traditional method. Similarly, IGNN uses 18.5% more memory to run on the CPU than the traditional method but takes only 2.9% of the time. In general, although the memory usage of IGNN is slightly higher compared to traditional methods, this cross-order of magnitude speedup shows the extraordinarily higher computational efficiency of IGNN.

5. Conclusions

- (1) In this study, we propose a neural network model, IGNN, for generating geometrically adaptive meshes. IGNN is based on the theory of generative adversarial learning to achieve accurate mapping between geometry and its corresponding MSF.

- (2) The neural network takes the SDF as input and generates an MSF represented as a Cartesian grid that contains the local mesh size or spatial resolution of the mesh. Based on the predicted MSF, a mesh aligned to the geometric features is then constructed over the domain.
- (3) We designed a polygonal dataset to validate the optimal choice of model parameters and structure. Furthermore, we constructed a spline geometry dataset as well as an airfoil dataset to evaluate the generalization of the optimal model with respect to its practical applicability.
- (4) The experimental results demonstrate that the proposed method effectively captures the geometric features of the domain to be meshed, thereby generating geometrically adaptive meshes that are comparable to those generated from the traditional methods.

Our work indicates that machine learning techniques can be used to achieve automatic mesh generation by learning from mesh data, significantly decreasing the workload of manual parameter specification in multiple steps of the traditional methods.

The training data with a certain amount for the proposed method poses a challenge for model training, which is attributed to the fact that model training requires a comprehensive dataset to effectively represent complex geometric features. Future work will design strategies to mitigate the data requirements and improve the adaptability and efficiency of the method in different practical applications. As an implicit geometric representation, SDF can represent local and global geometric features well, but it may not be able to accurately represent some extremely irregular and very complex geometry features. More accurate and efficient strategies, such as the combination of parameter representation and feature representation, will be developed in the future to further improve the robustness of the model. Experiments on NACA0012, RAE2822 demonstrated the generalization of IGNN to single-element airfoils, and future work will extend IGNN to more complex geometries, i.e., from single-element airfoils to multi-element airfoils. In viscous flow simulations, the strong anisotropy in the boundary layer region requires the generation of anisotropic meshes, and subsequent work will extend the IGNN from isotropic mesh generation to the generation of mixed anisotropic and isotropic meshes. IGNN has been more effective in 2D mesh generation, but 3D meshes have more complex topology and local features, so it is necessary to extend IGNN to 3D mesh in future work.

CRedit authorship contribution statement

Ran XU: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Conceptualization. **Hongqiang LYU:** Supervision, Resources, Project administration, Formal analysis, Conceptualization. **Jian YU:** Writing – review & editing, Visualization, Validation, Formal analysis, Conceptualization. **Chenyu BAO:** Visualization, Validation, Software, Investigation, Formal analysis. **Hongfei WANG:** Visualization, Investigation, Formal analysis, Conceptualization. **Yufei LIU:** Visualization, Validation, Formal analysis, Conceptualization. **Xuejun LIU:** Writing – review

& editing, Writing – original draft, Validation, Resources, Methodology, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This study was co-supported by the Aeronautical Science Foundation of China (Nos. 2018ZA52002 and 2019ZA052011).

References

- Mavriplis DJ. Unstructured-mesh discretizations and solvers for computational aerodynamics. *AIAA J* 2008;**46**(6):1281–98.
- Pirzadeh SZ. Advanced unstructured grid generation for complex aerodynamic applications. *AIAA J* 2010;**48**(5):904–15.
- Ruppert J. A Delaunay refinement algorithm for quality 2-dimensional mesh generation. *J Algorithms* 1995;**18**(3):548–85.
- Shewchuk JR. Triangle: Delaunay refinement algorithms for triangular mesh generation. *Comput Geom* 2002;**22**(1–3):21–74.
- George PL, Seveno E. The advancing-front mesh generation method revisited. *Int J Numer Methods Eng* 1994;**37**(21):3605–19.
- Ye H, Chen J, Pang Y, et al. Fast advancing layer method for viscous mesh generation. *Chin J Aeronaut* 2023;**36**(9):133–50.
- Fischer A, Bar-Yoseph PZ. Adaptive mesh generation based on multi-resolution quadtree representation. *Int J Numer Methods Eng*. 2000;**48**(11):1571–82.
- Bawin A, Henrotte F, Remacle JF. Automatic feature-preserving size field for three-dimensional mesh generation. *Int J Numer Methods Eng* 2021;**122**(18):4825–47.
- Persson PO. Mesh size functions for implicit geometries and PDE-based gradient limiting. *Eng Comput* 2006;**22**(2):95–109.
- Chen JJ, Liu ZW, Zheng Y, et al. Automatic sizing functions for 3d unstructured mesh generation. *26th international meshing roundtable*. 2017;203. p. 245–57.
- Lock C, Hassan O, Sevilla R, et al. Meshing using neural networks for improving the efficiency of computer modelling. *Eng Comput* 2023;**39**(6):3791–820.
- Deister F, Tremel U, Hassan O, et al. Fully automatic and fast mesh size specification for unstructured mesh generation. *Eng Comput* 2004;**20**(3):237–48.
- Pirzadeh S. Structured background grids for generation of unstructured grids by advancing-front method. *AIAA J* 1993;**31**(2):257–65.
- Quadros WR, Vyas V, Brewer M, et al. A computational framework for automating generation of sizing function in assembly meshing via disconnected skeletons. *Eng Comput* 2010;**26**(3):231–47.
- Wu H, Liu X, An W, et al. A generative deep learning framework for airfoil flow field prediction with sparse data. *Chin J Aeronaut* 2022;**35**(1):470–84.
- Sun Z, Wang C, Zheng Y, et al. Non-intrusive reduced-order model for predicting transonic flow with varying geometries. *Chin J Aeronaut* 2020;**33**(2):508–19.
- Li Y, Chang J, Kong C, et al. Recent progress of machine learning in flow modeling and active flow control. *Chin J Aeronaut* 2022;**35**(4):14–44.
- Gao H, Zahr MJ, Wang JX. Physics-informed graph neural galerkin networks: a unified framework for solving PDE-governed forward and inverse problems. *Comput Methods Appl Mech Eng* 2022;**390**:114502.
- Raissi M, Perdikaris P, Karniadakis GE. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J Comput Phys* 2019;**378**:686–707.
- Wang J, Li R, He C, et al. An inverse design method for supercritical airfoil based on conditional generative models. *Chin J Aeronaut* 2022;**35**(3):62–74.
- Ma Y, Du X, Sun X. Adaptive modification of turbofan engine nonlinear model based on LSTM neural networks and hybrid optimization method. *Chin J Aeronaut* 2022;**35**(9):314–32.
- Dyck DN, Lowther DA, McFee S. Determining an approximate finite element mesh density using neural network techniques. *IEEE Trans Magn* 1992;**28**(2):1767–70.
- Lowther DA, Dyck DN. A density driven mesh generator guided by a neural network. *IEEE Trans Magn* 1993;**29**(2):1927–30.
- Alfonzetti S, Coco S, Cavalieri S, et al. Automatic mesh generation by the let-it-grow neural network. *IEEE Trans Magn* 1996;**32**(3):1349–52.
- Alfonzetti S. A finite element mesh generator based on an adaptive neural network. *IEEE Trans Magn* 1998;**34**(5):3363–6.
- Alfonzetti S. A neural network generator for tetrahedral meshes. *IEEE Trans Magn* 2003;**39**(3):1650–3.
- Wang NH, Li P, Chang XH, et al. Unstructured mesh size control method based on artificial neural network. *Chin J Theor Appl Mech* 2021;**53**(10):2682–91.
- Zhang Z, Wang Y, Jimack PK, et al. MeshingNet: a new mesh generation method based on deep learning. In: Krzhizhanovskaya VV, Závodszy G, Lees MH, et al., editors. *Computational science – ICCS 2020*. Springer International Publishing; 2020. p. 186–98.
- Kim M, Lee J, Kim J. GMR-net: GCN-based mesh refinement framework for elliptic PDE problems. *Eng Comput* 2023;**39**(5):3721–37.
- Kipf TN, Welling M. Semi-supervised classification with graph convolutional networks. *arXiv preprint:1609.02907*; 2016.
- Chan CL, Scholz F, Takacs T. Locally refined quad meshing for linear elasticity problems based on convolutional neural networks. *Eng Comput* 2022;**38**(5):4631–52.
- Huang K, Krügener M, Brown A, et al. Machine learning-based optimal mesh generation in computational fluid dynamics. *arXiv prepr:2102.12923*; 2021.
- Goodfellow I, Pouget-Abadie J, Mirza M, et al. Generative adversarial networks. *Commun ACM* 2020;**63**(11):139–44.
- Osher S, Fedkiw R, Piechor K. Level set methods and dynamic implicit surfaces. *Appl Mech Rev* 2004;**57**(3):B15.
- Ren Y, Wang F, Zhang T, et al. VolRecon: volume rendering of signed ray distance functions for generalizable multi-view reconstruction. In: *2023 IEEE/CVF conference on computer vision and pattern recognition (CVPR)*. 2023. p. 16685–95.
- Sethian JA. A fast marching level set method for monotonically advancing fronts. *Proc Natl Acad Sci* 1996;**93**(4):1591–5.
- Gulrajani I, Ahmed F, Arjovsky M, et al. Improved training of wasserstein gans. *Adv Neural Inf Proces Syst* 2017.
- Radford A. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv prepr:1511.06434*; 2015.
- Mirza M. Conditional generative adversarial nets. *arXiv prepr:1411.1784*; 2014.
- Mao X, Li Q, Xie H, et al. Least squares generative adversarial networks. *Proceedings of the IEEE international conference on computer vision*; 2017. p. 2794–802.
- Isola P, Zhu JY, Zhou T, et al. Image-to-image translation with conditional adversarial networks. *2017 IEEE conference on computer vision and pattern recognition (CVPR)*; 2016. p. 5967–76.

42. Zhu JY, Park T, Isola P, et al. Unpaired image-to-image translation using cycle-consistent adversarial networks. In: *2017 IEEE international conference on computer vision (ICCV)*; 2017. p. 2242–51.
43. He K, Zhang X, Ren S, et al. Deep residual learning for image recognition. *2016 IEEE conference on computer vision and pattern recognition (CVPR)*; 2016. p. 770–8.
44. Johnson J, Alahi A, Li FF. Perceptual losses for real-time style transfer and super-resolution. In: Leibe B, Matas J, Sebe N, Welling M, eds. *Computer Vision - ECCV 2016. 14th European conference. Proceedings: LNCS 9906*; 2016. p. 694–711.
45. Logg A, Mardal KA, Wells G. *Automated solution of differential equations by the finite element method: the FEniCS book*. Springer; 2012.
46. Geuzaine C, Remacle JF. Gmsh: a 3-D finite element mesh generator with built-in pre- and post-processing facilities. *Int J Numer Methods Eng* 2009;**79**(11):1309–31.